





# Rapport projet d'Algorithmie des graphes

Résolution du problème du voyageur de commerce

Tristan GROULT, Louis GUERAIN, Kibandou BIGNANG

2 janvier 2026

# Sommaire

**1 Notice d'utilisation** **3**

**2 Algorithme** **3**

2.1 Point le Plus Proche (PPP) . . . . . 3
2.2 Point le Plus Proche Optimisé (OptPPP) . . . . . 6
2.3 Parcour en largeur d'un ACM à partir de Prim (OptPrim) . . . . . 7
2.4 OptPrim + Décroisé (OptOptPrim) . . . . . 11
2.5 Évaluation et séparation progressive ("branch and bound") (HDS) pas fait . . 11

# 1 Notice d'utilisation

Ce projet dépend des bibliothèques `numpy` et d'autres modules standards. Il dépend aussi de `flask` pour l'application web.

```
pip install numpy # (tout)
pip install flask # (web app)
```

Listing 1 – Installation des dépendances

Ou plus simplement :

```
pip install requirement.txt
```

# 2 Algorithme

## 2.1 Point le Plus Proche (PPP)

**Algorithme : Procédure PPP (Point le Plus Proche)**
**Description :** Construit un cycle hamiltonien en insérant itérativement le point le plus proche du cycle actuel.
**Vars :**
- $c$ : Liste d'entiers (Cycle hamiltonien)
- $l$ : Réel (Longueur du cycle)
- $Q_i, Q_j$ : Entiers (Indices des points sélectionnés)

**Entrées :**
- $L$ : Liste de points $P_1, \ldots, P_n$
- $Q$ : Un point de départ choisi dans $L$
- $D$ : La matrice des distances $n \times n$

**Sorties :**
- $c$ : Un cycle hamiltonien
- $l$ : La longueur du cycle

**Début :**

1. $c \leftarrow$ cycle trivial composé du seul point $Q$
2. $l \leftarrow 0$
3. Retirer $Q$ de $L$
4. Tant que $L$ n'est pas vide faire :
5. $\quad (Q_i, Q_j) \leftarrow \texttt{TrouverCoupleMin}(L, c, D)$
6. $\quad (c, l) \leftarrow \texttt{InsérerDansCycle}(c, Q_i, Q_j, D, l)$
7. $\quad$ Retirer $Q_i$ de $L$
8. Retourner $c, l$

**Algorithme : InsérerDansCycle**
**Description :** Insère $Q_i$ dans $c$ soit entre $Q_j$ et son successeur, soit entre son prédécesseur et $Q_j$, selon le coût minimal, et met à jour la longueur du cycle.
**Vars :**
— $Q_{next}, Q_{prev}$ : Entiers (Indices des voisins de $Q_j$)
— $\text{coût}_{\text{droite}}, \text{coût}_{\text{gauche}}$ : Réels (Coûts d'insertion)
**Entrées :**
— $c$ : Cycle partiel
— $Q_i$ : Point à insérer
— $Q_j$ : Point d'ancrage dans $c$
— $D$ : Matrice des distances
— $l$ : Longueur actuelle du cycle
**Sorties :** $c$ (cycle mis à jour), $l$ (nouvelle longueur)
**Début :**

1. Soit $Q_{next}$ le successeur de $Q_j$ dans $c$
2. Soit $Q_{prev}$ le prédécesseur de $Q_j$ dans $c$
3. $\text{coût}_{\text{droite}} \leftarrow D[Q_j, Q_i] + D[Q_i, Q_{next}] - D[Q_j, Q_{next}]$
4. $\text{coût}_{\text{gauche}} \leftarrow D[Q_{prev}, Q_i] + D[Q_i, Q_j] - D[Q_{prev}, Q_j]$
5. Si $\text{coût}_{\text{droite}} \leq \text{coût}_{\text{gauche}}$ alors :
6. Insérer $Q_i$ entre $Q_j$ et $Q_{next}$ dans $c$
7. $l \leftarrow l + \text{coût}_{\text{droite}}$
8. Sinon :
9. Insérer $Q_i$ entre $Q_{prev}$ et $Q_j$ dans $c$
10. $l \leftarrow l + \text{coût}_{\text{gauche}}$
11. Retourner $c, l$

**Algorithme : TrouverCoupleMin**
**Description :** Trouve le couple $(Q_i, Q_j)$ tel que $Q_i \in L$, $Q_j \in c$ et la distance $D[Q_i, Q_j]$ est minimale.
**Vars :**
— $d_{min}$ : Réel (Distance minimale trouvée)
— $u, v$ : Entiers (Indices de parcours)
**Entrées :**
— $L$ : Liste des points non encore visités
— $c$ : Le cycle partiel actuel
— $D$ : La matrice des distances $n \times n$
**Sorties :** $Q_i$ (point sélectionné dans $L$), $Q_j$ (point d'ancrage dans $c$)
**Début :**

1. $d_{min} \leftarrow \infty$
2. $Q_i \leftarrow$ vide
3. $Q_j \leftarrow$ vide
4. Pour chaque point $u$ appartenant à $L$ faire :
5. Pour chaque point $v$ appartenant à $c$ faire :
6. Si $D[u, v] < d_{min}$ alors :
7. $d_{min} \leftarrow D[u, v]$
8. $Q_i \leftarrow u$
9. $Q_j \leftarrow v$
10. Retourner $Q_i, Q_j$

## 2.2 Point le Plus Proche Optimisé (OptPPP)

**Algorithme : Procédure Décroise**
**Description :** Améliore un cycle en décroisant les arêtes tant que cela réduit la distance totale.
**Vars :**
- $n$ : Entier (Nombre de points)
- amélioration : Booléen (Indicateur de modification)
- $i, j$ : Entiers (Indices de boucle)
- $u, v, x, y$ : Entiers (Indices des points des arêtes)
- $d_{actuelle}, d_{nouvelle}$ : Réels (Distances comparées)
- $p, q$ : Entiers (Indices pour l'inversion)

**Entrées :**
- $c$ : Cycle hamiltonien initial
- $D$ : Matrice des distances

**Sorties :** $c$ (cycle optimisé), $l$ (longueur)
**Début :**

1. $n \leftarrow$ taille$(c)$
2. amélioration $\leftarrow$ Vrai
3. Tant que amélioration faire :
4. amélioration $\leftarrow$ Faux
5. Pour $i$ allant de $1$ à $n-2$ faire :
6. Pour $j$ allant de $i+2$ à $n$ faire :
7. $u \leftarrow c[i]$
8. $v \leftarrow c[i+1]$
9. $x \leftarrow c[j]$
10. Si $j = n$ alors :
11. $y \leftarrow c[1]$
12. Sinon :
13. $y \leftarrow c[j+1]$
14. $d_{actuelle} \leftarrow D[u, v] + D[x, y]$
15. $d_{nouvelle} \leftarrow D[u, x] + D[v, y]$
16. Si $d_{nouvelle} < d_{actuelle}$ alors :
17. $c[i+1 \dots j] \leftarrow$ inverser$(c[i+1 \dots j])$
18. amélioration $\leftarrow$ Vrai
19. $l \leftarrow$ CalculerLongueur$(c, D)$
20. Retourner $c, l$

**Algorithme : Procédure OptPPP**
**Description :** Wrapper pour l'algorithme de décroisement.
**Vars :** Aucune
**Entrées :**
— $c$ : Cycle hamiltonien initial
— $D$ : Matrice des distances
**Sorties :** $c$ (cycle optimisé), $l$ (longueur)
**Début :**
1. Retourner `Décroise`$(c, D)$

## 2.3 Parcour en largeur d'un ACM à partir de Prim (OptPrim)

**Structure de données : GraphePrim**
**Type** GraphePrim = **Enregistrement**
cle : Tableau[1..n] de Réel ;
pred : Tableau[1..n] d'Entier ;
NoeudTas : Tableau[1..n] d'Entier
Tas : Tableau[1..n] d'Entier
1 **Fin** ;

**Algorithme : Prim (Optimisé)**
**Description :** Construit un arbre couvrant de poids minimum (MST) en utilisant un tas binaire pour sélectionner efficacement le prochain sommet le plus proche.
**Vars :**
— $GraphePrim$ : Structure GraphePrim (Données du graphe et du tas)
— $pres$ : Tableau de booléens (Indique si un sommet est dans le tas)
— $u, v$ : Entiers (Indices de sommets)
**Entrées :**
— $D$ : Matrice des distances $n \times n$
— $r0$ : Sommet de départ (par défaut 1)
**Sorties :**
— $pred$ : Liste des prédécesseurs définissant l'arbre couvrant
**Début :**

1. Initialiser $GraphePrim$ avec $n$ sommets
2. $pres \leftarrow$ Tableau de booléens initialisé à `Vrai` (indique si un sommet est dans le tas)
3. $GraphePrim.cle[r0] \leftarrow 0$
4. $GraphePrim.echanger(1, GraphePrim.noeudTas[r0])$ // *Placer r0 à la racine*
5. Tant que $GraphePrim.tas$ n'est pas vide faire :
6. $u \leftarrow GraphePrim.tas[1]$ // *Extraire le min*
7. $pres[u] \leftarrow$ `Faux`
8. $GraphePrim.echanger(1, \text{taille}(tas))$
9. Retirer le dernier élément de $GraphePrim.tas$
10. Si $GraphePrim.tas$ non vide alors :
11. $GraphePrim.versLeBas(1)$
12. Pour chaque sommet $v$ de 1 à $n$ faire :
13. Si $pres[v]$ et $D[u, v] < GraphePrim.cle[v]$ alors :
14. $GraphePrim.cle[v] \leftarrow D[u, v]$
15. $GraphePrim.pred[v] \leftarrow u$
16. $GraphePrim.versLeHaut(GraphePrim.noeudTas[v])$
17. Retourner $GraphePrim.pred$

**Algorithme : PredToTree**
**Description :** Convertit la liste des prédécesseurs en une structure d'arbre (dictionnaire d'adjacence).
**Vars :**
— $d$ : Dictionnaire (Liste d'adjacence)
— $racine$ : Entier (Indice de la racine)
— $i$ : Entier (Indice de boucle)
**Entrées :**
— $pred$ : Liste des prédécesseurs
**Sorties :** $d$ (arbre), $racine$
**Début :**

1. $d \leftarrow$ Dictionnaire vide
2. $racine \leftarrow -1$
3. Pour $i$ allant de $1$ à taille$(pred)$ faire :
4. $\quad d[i] \leftarrow$ Liste vide
5. $\quad$ Si $pred[i]$ est `None` ou $-1$ alors :
6. $\quad\quad racine \leftarrow i$
7. Pour $i$ allant de $1$ à taille$(pred)$ faire :
8. $\quad$ Si $pred[i]$ n'est pas `None` et $pred[i] \neq -1$ alors :
9. $\quad\quad$ Ajouter $i$ à $d[pred[i]]$
10. Retourner $d, racine$

**Algorithme : ParcoursPrefixe**
**Description :** Effectue un parcours en profondeur (DFS) pour obtenir l'ordre de visite des sommets.
**Vars :**
- $stack$ : Pile d'entiers
- $p$ : Tableau d'entiers (Marquage visite)
- $p_etoile$ : Liste d'entiers (Résultat)
- $count$ : Entier
- $lst_voisin$ : Liste d'entiers
- $rep$ : Booléen
- $voisin$ : Entier

**Entrées :**
- $d$ : Dictionnaire d'adjacence
- $racine$ : Entier

**Sorties :** $p_etoile$ (liste ordonnée)
**Début :**

1. $stack \leftarrow [racine]$
2. $p \leftarrow$ Tableau de zéros de taille $n$
3. $p_etoile \leftarrow [racine]$
4. $p[racine] \leftarrow 1$
5. $count \leftarrow 2$
6. Tant que $stack$ n'est pas vide faire :
7. $\quad lst_voisin \leftarrow d[\text{sommet}(stack)]$
8. $\quad rep \leftarrow$ Faux
9. $\quad$ Pour chaque $voisin$ dans $lst_voisin$ faire :
10. $\quad\quad$ Si $p[voisin] == 0$ alors :
11. $\quad\quad\quad$ Empiler $voisin$ sur $stack$
12. $\quad\quad\quad$ Ajouter $voisin$ à $p_etoile$
13. $\quad\quad\quad p[voisin] \leftarrow count$
14. $\quad\quad\quad count \leftarrow count + 1$
15. $\quad\quad\quad rep \leftarrow$ Vrai
16. $\quad\quad\quad$ Sortir de la boucle Pour
17. $\quad$ Si non $rep$ alors :
18. $\quad\quad$ Dépiler de $stack$
19. Retourner $p_etoile$

**Algorithme : Procédure OptPrim**
**Description :** Construit un cycle hamiltonien à partir de l'arbre couvrant minimal.
**Vars :**
- $pred$ : Liste d'entiers
- $d$ : Dictionnaire
- $racine$ : Entier
- $c$ : Liste d'entiers
- $l$ : Réel

**Entrées :**
- $points$ : Liste de points
- $D$ : Matrice des distances
- $r0$ : Sommet de départ

**Sorties :** $c, l$
**Début :**

1. Si $D$ est `None` alors :
2. $\quad D \leftarrow$ `CalculerMatriceDistances`$(points)$
3. $pred \leftarrow$ `Prim`$(D, r0)$
4. $(d, racine) \leftarrow$ `PredToTree`$(pred)$
5. $c \leftarrow$ `ParcoursPrefixe`$(d, racine)$
6. $l \leftarrow$ `CalculerLongueur`$(c, D)$
7. Retourner $c, l$

## 2.4 OptPrim + Décroisé (OptOptPrim)

**Algorithme : Procédure OptOptPrim**
**Description :** Combine OptPrim et le décroisement.
**Vars :**
- $c_{init}$ : Liste d'entiers (Cycle initial)

**Entrées :**
- $points$ : Liste de points
- $D$ : Matrice des distances
- $r0$ : Sommet de départ

**Sorties :** $c, l$
**Début :**

1. Si $D$ est `None` alors :
2. $\quad D \leftarrow$ `CalculerMatriceDistances`$(points)$
3. $(c_{init}, _) \leftarrow$ `OptPrim`$(points, D, r0)$
4. Retourner `Décroise`$(c_{init}, D)$

## 2.5 Évaluation et séparation progressive ("branch and bound") (HDS) pas fait

*Section à compléter.*